
AgentTrace

Open-Source Observability Platform
for Multi-Agent AI Systems

"See exactly what your agents are doing —
every step, every decision, every cost."

AUTHOR	Umair — University of Lahore, Pakistan
GITHUB	github.com/umairinayat
VERSION	v0.1 Roadmap
LICENSE	MIT
DATE	March 2026

Table of Contents

- | | |
|---|--|
| 1 | Project Overview
What AgentTrace is, the problem it solves, and who it is for |
| 2 | Architecture Overview
System components, data flow, and technology stack |
| 3 | Cost Analysis
Zero-cost development breakdown and infrastructure plan |
| 4 | Detailed Phase Roadmap
Week-by-week implementation plan across 7 phases |
| 5 | Full Timeline Summary
Consolidated timeline from setup to research paper |
| 6 | GitHub Repository Best Practices
README structure, labels, and initial issues |
| 7 | Zero-Cost Infrastructure Setup
Development environment and free cloud services |
| 8 | Success Metrics
3-month goals and what success looks like |

1. Project Overview

1.1 What is AgentTrace?

AgentTrace is a fully open-source, self-hosted observability and debugging platform for multi-agent AI systems. It captures every LLM call, tool execution, sub-agent message, token cost, and latency across your entire agent pipeline — then correlates them into a visual trace timeline.

It also detects **behavioral drift**: when your agent silently starts behaving differently after a model provider updates their backend. This is the core research differentiator that separates AgentTrace from every other observability tool in the ecosystem.

1.2 The Problem It Solves

- **89% of developers** believe they have agent observability — but only **62%** can actually monitor agent actions step by step (industry survey, 1,300+ developers, 2026)
- Multi-agent traces in existing tools (LangSmith, Langfuse) frequently **merge or lose correlation** across sub-agents, making debugging multi-agent workflows extremely difficult
- **No open-source tool detects behavioral drift over time** — when a model provider silently updates their backend, your agent's behavior changes without any code changes on your end
- Commercial solutions (Arize AI, Galileo) are **expensive and cloud-only**, creating vendor lock-in and data privacy concerns
- OpenTelemetry semantic conventions for AI agents are **still unstable** (the GenAI SIG is in alpha), leaving developers without a standard tracing framework

1.3 Who Uses AgentTrace?

- **Multi-agent developers** building systems with CrewAI, LangGraph, AutoGen, OpenAI Agents SDK, or custom pipelines who need end-to-end trace visibility
- **ML researchers** studying agent behavior patterns, decision-making sequences, and tool usage across different model configurations
- **Production teams** deploying AI agents who need audit trails, cost tracking, and latency monitoring to maintain SLAs
- **Compliance teams** at companies that must meet EU AI Act observability requirements, which mandate logging and explainability for high-risk AI systems

1.4 What Makes It Unique?

Three differentiators versus existing tools:

Differentiator	Description	Competitors
Behavioral Drift Detection	Detects when model providers silently update their backends and your agent changes behavior. Uses embedding cosine similarity + statistical distribution tests.	None offer this
Fully Self-Hosted	One <code>docker compose up</code> command. No cloud dependency, no API keys required to run AgentTrace itself. Your data stays on your machine.	LangSmith: cloud only Langfuse: self-host available Arize: cloud only
Zero-Dependency SDK	Two lines of Python to start tracing. No OpenTelemetry configuration, no complex setup, no vendor SDKs. Just <code>pip install agenttrace</code> .	All competitors require extensive configuration

2. Architecture Overview

2.1 High-Level Architecture

AgentTrace is composed of six core components that work together to provide end-to-end observability for multi-agent AI systems. Each component is independently deployable and communicates via well-defined interfaces.

Component 1 — Python SDK (`agenttrace-sdk`)

The SDK is the entry point for all tracing. It provides lightweight decorators and context managers that instrument your agent code with zero performance overhead. The SDK hooks into LangChain, LangGraph, CrewAI, and AutoGen via monkey-patching — no changes to your existing agent code required.

The SDK collects:

- Prompt text and response text (full or truncated, configurable)
- Tool calls: function name, arguments, return values, and latency
- Token counts: `prompt_tokens`, `completion_tokens`, `total_tokens`
- Cost estimate based on public pricing tables (updated via JSON config)
- Agent ID, parent trace ID, and span hierarchy for multi-agent correlation

All data is sent to the Collector via **async HTTP** (non-blocking) — your agent's performance is never impacted by the tracing overhead.

Component 2 — Collector Service (FastAPI)

The Collector is a FastAPI backend that receives trace events from the SDK, validates and enriches them (adds server timestamps, calculates costs from the pricing table), and writes them to the storage backend. It also exposes a REST API that the Dashboard consumes for visualization and querying.

Component 3 — Trace Storage

AgentTrace supports two storage backends depending on deployment context:

- **SQLite** for local development — zero setup, single file, included in Python's standard library

- **PostgreSQL + TimescaleDB** for production — time-series optimized queries, efficient retention policies, and horizontal scaling

The data model follows a hierarchical structure: **Trace** → **Spans** → **Events**. A Trace represents a complete agent execution. Each Trace contains multiple Spans (LLM calls, tool calls, sub-agent invocations). Each Span contains Events (start, end, error, metadata).

Component 4 — Drift Detector (Python Background Service)

The Drift Detector is the research differentiator. It runs as a background worker that continuously monitors agent behavior patterns and flags anomalies when current behavior deviates from a statistical baseline.

- Computes rolling statistical baseline of agent response patterns
- Uses **embedding cosine similarity** (sentence-transformers) to detect semantic drift
- Applies **Kolmogorov-Smirnov tests** on response length and token distributions
- Monitors tool call frequency changes
- Uses a local embedding model (all-MiniLM-L6-v2, 80MB) — no API key needed

Component 5 — Dashboard (React + D3.js)

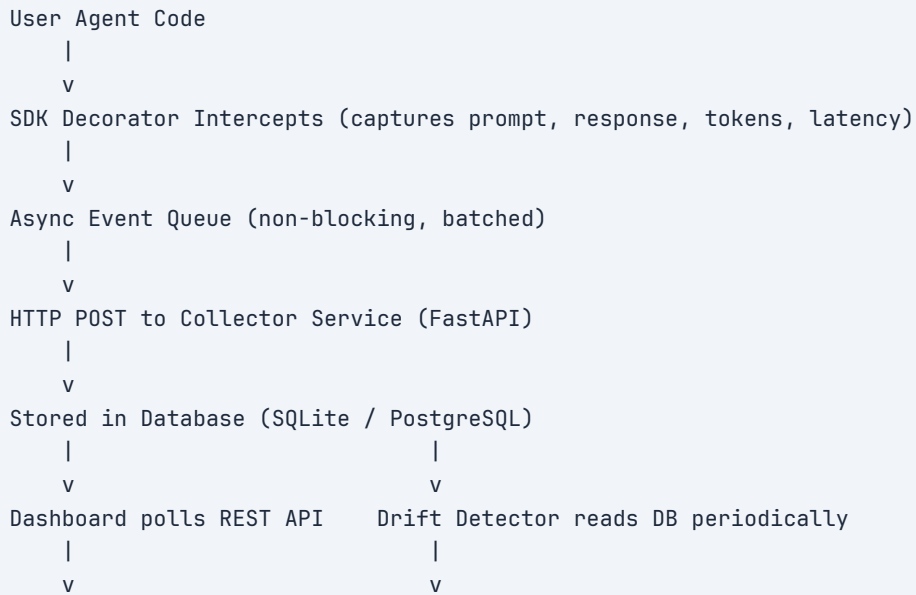
- Visual trace timeline (Gantt-style) showing every span in an agent execution
- Token cost per step breakdown with daily/weekly aggregation
- Latency heatmap across agents and models
- Behavioral drift chart (time series) with configurable alert thresholds
- Real-time alert feed with filtering and search

Component 6 — Alert System

- Webhook support: Slack, Discord, and custom HTTP endpoints
- Email alerts via configurable SMTP
- Configurable thresholds per agent and per metric

2.2 Data Flow

Data Flow Diagram



2.3 Technology Stack

Component	Technology	Rationale
SDK	Python 3.11+, asyncio, httpx	Lightweight, non-blocking, native async
Collector	FastAPI, Pydantic v2, SQLAlchemy	Fast async API, type-safe validation
Storage (dev)	SQLite	Zero setup, built into Python
Storage (prod)	PostgreSQL + TimescaleDB	Time-series optimized, scalable
Drift Detector	sentence-transformers, numpy, scipy	Runs locally, no API needed
Dashboard	React 18, D3.js, TailwindCSS	Fast interactive UI, custom visualizations
Deployment	Docker Compose	One command setup
Embedding Model	all-MiniLM-L6-v2 (local)	80MB, runs on CPU, zero cost

3. Cost Analysis

3.1 Do You Need to Spend Money?

Answer: No. Zero required spending.

AgentTrace is designed to be built, tested, and deployed entirely for free. Every component uses open-source tools with generous free tiers. The table below details every resource you might need and its zero-cost alternative.

Resource	Required ?	Free Alternative	Notes
LLM API (OpenAI, Anthropic)	NO	Ollama + local models (Qwen3, Llama 4)	AgentTrace traces ANY LLM, local or cloud
Embedding model for drift	NO	sentence-transformers (local)	80MB model, runs on CPU
Database	NO	SQLite (built-in)	Upgrade to PostgreSQL later for free
Dashboard hosting	NO	Runs on localhost:3000	Docker Compose included
CI/CD	NO	GitHub Actions (free for public repos)	2,000 minutes/month free
Package hosting	NO	PyPI (free)	pip install agenttrace
Documentation	NO	GitHub Pages (free)	MkDocs Material theme
Domain name	OPTIONAL	GitHub Pages URL works fine	\$10/year if desired
Cloud demo hosting	OPTIONAL	Render.com free tier	Free 750 hours/month

3.2 When Would You Spend Money?

Only if you want to run a hosted cloud demo for the community — estimated **\$5–10/month** on Render or Railway. This is completely optional and not required for development, testing, or initial community launch.

3.3 Hardware You Already Have

Your **RTX 4090 (24GB VRAM)** is more than sufficient to run the entire development and testing stack:

- Run local LLMs for testing via Ollama: Qwen3 14B (~10GB VRAM), Llama 4 Scout (~16GB VRAM), Mistral 7B (~5GB VRAM)
- Run embedding models for drift detection (sentence-transformers uses CPU or a fraction of GPU memory)
- Develop and test the full stack simultaneously (collector, dashboard, drift detector)

Important: No GPU is actually required to run AgentTrace itself — it is a tracing and observability tool, not a model inference engine. The GPU is only used for running local LLMs during testing.

4. Detailed Phase Roadmap

PHASE 0

Phase 0: Project Setup (Week 1)

Goal: Repository structure, tooling, and CI/CD before writing any feature code. A professional foundation that signals quality from the first commit.

Tasks

- Create GitHub repository: `agenttrace/agenttrace` (organization + repo)
- Set up monorepo structure (see directory layout below)
- Set up GitHub Actions: lint (ruff), type check (mypy), tests (pytest), auto-publish to PyPI on tag
- Write README with badges, installation instructions, and a 30-second quickstart code block
- Set up issue templates: Bug Report, Feature Request, Integration Request
- Add CONTRIBUTING.md with clear guidelines for first-time contributors
- Add MIT LICENSE file

Repository Structure

Directory Layout

```
agenttrace/
├── sdk/                                # Python SDK (pip package)
│   ├── agenttrace/
│   │   ├── __init__.py
│   │   ├── tracer.py                # Core tracer class
│   │   ├── decorators.py           # @trace_agent decorator
│   │   ├── integrations/           # LangChain, CrewAI, AutoGen hooks
│   │   └── client.py               # HTTP client to send events
│   ├── tests/
│   └── pyproject.toml
├── collector/                          # FastAPI backend
│   ├── app/
│   │   ├── main.py
│   │   ├── models.py
│   │   ├── routes/
│   │   └── db/
│   ├── tests/
│   └── Dockerfile
├── dashboard/                          # React frontend
│   ├── src/
│   └── Dockerfile
├── drift-detector/                     # Python background worker
│   ├── detector.py
│   └── Dockerfile
├── docker-compose.yml
├── docker-compose.dev.yml
├── README.md
├── CONTRIBUTING.md
└── LICENSE (MIT)
```

Deliverable: Empty but fully structured repo with CI passing. First commit pushed.

PHASE 1

Phase 1: Core SDK (Weeks 2–3)

Goal: A working Python SDK that can trace any LLM call and send it to a collector. This is the foundation — everything else builds on this.

1.1 Core Tracer Class

Usage is designed to be minimal — two lines to start tracing:

SDK Usage Example

```
from agenttrace import tracer

tracer.init(collector_url="http://localhost:8000")

@tracer.trace_agent(name="my_agent")
def my_agent(query: str) -> str:
    response = llm.invoke(query)
```

The `tracer.init()` call performs three actions:

- Creates a background async queue for non-blocking event collection
- Initializes the HTTP client (httpx async) for sending events to the collector
- Sets global trace context using Python's contextvars for thread-safe correlation

The `@trace_agent` decorator captures:

- Function name, arguments (sanitized to remove secrets), and return value
- Wall clock time (latency) with high-resolution monotonic timer
- A unique `trace_id` (UUID4) and parent `trace_id` for nested agent correlation
- Thread/async context for proper correlation in concurrent workloads

1.2 LLM Call Interception

For LangChain integration, AgentTrace implements a custom `BaseCallbackHandler` that hooks into every LLM call and tool execution:

LangChain Callback Handler

```
from langchain.callbacks.base import BaseCallbackHandler

class AgentTraceCallback(BaseCallbackHandler):
    def on_llm_start(self, serialized, prompts, **kwargs):
        # Capture start time, prompt text, model name
        self._start_time = time.monotonic()
        self._prompt = prompts[0] if prompts else ""

    def on_llm_end(self, response, **kwargs):
        # Capture end time, response, token counts
        # Calculate latency_ms and estimated cost
        # Send event to collector via async queue
        latency = (time.monotonic() - self._start_time) * 1000
        self._send_span(event_type="llm_call", latency_ms=latency)

    def on_tool_start(self, serialized, input_str, **kwargs):
        # Capture tool name, input arguments
        pass

    def on_tool_end(self, output, **kwargs):
        # Capture tool output, calculate latency
```

Data captured per LLM call:

- Model name (e.g., gpt-4o, claude-3.5-sonnet, qwen3:14b)
- Full prompt and response text
- Token usage: prompt_tokens, completion_tokens, total_tokens
- Latency in milliseconds
- Estimated cost (based on public pricing tables, updated via JSON config file)

1.3 Event Data Model

All trace events conform to a strict Pydantic schema:

Pydantic Event Model

```
class SpanEvent(BaseModel):
    trace_id: str                # UUID4
    span_id: str                 # UUID4
    parent_span_id: Optional[str] # For nested agents
    agent_name: str
    event_type: Literal[
        "llm_call", "tool_call", "agent_start",
        "agent_end", "message"
    ]
    timestamp: datetime
    latency_ms: Optional[float]
    model: Optional[str]
    prompt_tokens: Optional[int]
    completion_tokens: Optional[int]
    cost_usd: Optional[float]
    input_data: Optional[dict]
    output_data: Optional[dict]
```

1.4 Async Event Queue

All events are non-blocking. The SDK never slows down your agent:

- Events go into an asyncio queue (non-blocking producer)
- A background coroutine batches events and sends via httpx
- If collector is unavailable: events are buffered to a local SQLite file (no data loss)
- Configurable batch size (default: 10 events) and flush interval (default: 2 seconds)

1.5 Framework Integrations (Phase 1 Scope)

Phase 1 supports the three most common frameworks:

- **LangChain** — via custom CallbackHandler (shown above)
- **LangGraph** — via StateGraph middleware that intercepts node transitions
- **Raw OpenAI / Anthropic / Ollama API** — via httpx monkey-patch on the HTTP transport

Phase 2+ adds: CrewAI, AutoGen, OpenAI Agents SDK, and a manual span API for custom agents.

Deliverable: `pip install agenttrace` working. LangChain traces captured and sent to collector.

Tests to Write

- Unit tests for event serialization (Pydantic model validation)
- Unit tests for the async queue (verify non-blocking behavior)
- Integration test: trace a simple LangChain chain end-to-end

PHASE 2

Phase 2: Collector Service (Weeks 3–4)

Goal: A FastAPI backend that receives, stores, and serves trace data. The backbone of the entire observability pipeline.

2.1 Database Schema (SQLAlchemy Async)

SQL Schema

```
-- traces table
CREATE TABLE traces (
  id TEXT PRIMARY KEY,          -- UUID
  name TEXT NOT NULL,           -- agent pipeline name
  started_at TIMESTAMP NOT NULL,
  ended_at TIMESTAMP,
  total_tokens INTEGER,
  total_cost_usd REAL,
  status TEXT,                  -- running, completed, error
  metadata JSONB
);

-- spans table
CREATE TABLE spans (
  id TEXT PRIMARY KEY,
  trace_id TEXT REFERENCES traces(id),
  parent_span_id TEXT,
  agent_name TEXT,
  event_type TEXT,
  started_at TIMESTAMP,
  ended_at TIMESTAMP,
  latency_ms REAL,
  model TEXT,
  prompt_tokens INTEGER,
  completion_tokens INTEGER,
  cost_usd REAL,
  input_data JSONB,
  output_data JSONB,
  metadata JSONB
);

-- drift_baselines table (populated in Phase 3)
CREATE TABLE drift_baselines (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  agent_name TEXT,
  window_start TIMESTAMP,
  window_end TIMESTAMP,
  avg_response_length REAL,
  avg_latency_ms REAL,
  avg_token_count REAL,
  embedding_centroid BLOB,      -- serialized numpy array
  tool_call_distribution JSONB
);
```

2.2 REST API Endpoints

Method	Path	Description
POST	/api/v1/spans	Ingest batch of span events from SDK
GET	/api/v1/traces	List all traces (paginated, filterable by date/agent/status)
GET	/api/v1/traces/{trace_id}	Get full trace with all spans (hierarchical)
GET	/api/v1/traces/{trace_id}/timeline	Get Gantt-format timeline data for visualization
GET	/api/v1/stats	Aggregate stats (total cost, avg latency, trace count)
GET	/api/v1/drift/alerts	Get drift alerts (paginated, filterable)
GET	/api/v1/health	Health check endpoint

2.3 Cost Estimation Engine

The collector maintains a `pricing.json` file with current model pricing. Cost is calculated per span using the formula:

Cost Calculation

```
cost = (prompt_tokens / 1000) * input_price_per_1k  
      + (completion_tokens / 1000) * output_price_per_1k
```

The pricing file can be updated manually or via cron job. Community contributions keep pricing current.

2.4 Deployment

- Dockerfile for the collector (Python 3.11-slim base, ~80MB image)
- `docker-compose.yml` includes: collector + SQLite volume mount
- Optional `docker-compose.prod.yml` with PostgreSQL + TimescaleDB

Deliverable: `docker compose up` starts the full backend. Traces visible via REST API. Swagger docs auto-generated at `/docs`.

PHASE 3

Phase 3: Behavioral Drift Detector (Weeks 4–5)

Goal: Detect when your agent silently changes behavior — the key research differentiator that makes AgentTrace unique in the ecosystem.

3.1 What is Behavioral Drift?

Model providers (OpenAI, Anthropic, Google) silently update their model backends on a regular basis. A system prompt that produced reliable, predictable behavior in January may behave subtly differently by March — not because your code changed, but because the model did.

This is **undetectable without a behavioral baseline**. Traditional monitoring (latency, error rates) will not catch semantic changes in agent responses. You need to compare the meaning of responses over time, not just their surface statistics.

3.2 Drift Detection Algorithm

Step 1 — Build Baseline

- For each agent, collect response texts from the last N traces (default: N=50)
- Compute embeddings for all responses using `sentence-transformers/all-MiniLM-L6-v2` (runs locally, no API key)
- Store the centroid embedding vector as the behavioral baseline
- Record: average response length, average latency, average token count, tool call frequency distribution

Step 2 — Ongoing Monitoring

- Every new batch of K traces (default: K=10), compute the same metrics
- Compare to baseline using four complementary statistical methods

Metric	Method	Alert Threshold
Semantic similarity	Cosine distance of response embeddings vs. baseline centroid	> 0.15 (configurable)
Response distribution	Kolmogorov-Smirnov test on response length distribution	p-value < 0.05
Token usage	Relative change in average token count	> 20% change

Metric	Method	Alert Threshold
Tool call patterns	Relative change in tool call frequency	> 20% change

Step 3 — Alert Generation

When any threshold is breached, the drift detector stores an alert in the database and sends it via the configured webhook (Slack, Discord, or custom). Alerts include the specific metric that triggered, the measured value vs. the threshold, and a link to the relevant traces in the dashboard.

3.3 Code Structure

Drift Detector Core Class

```
class DriftDetector:
    def __init__(self, db_url: str,
                  embedding_model: str = "all-MiniLM-L6-v2"):
        self.embedder = SentenceTransformer(embedding_model)
        self.db = Database(db_url)

    def build_baseline(self, agent_name: str,
                      n_traces: int = 50):
        # Fetch last N response texts from DB
        # Compute embeddings via sentence-transformers
        # Calculate centroid + statistical baselines
        # Store in drift_baselines table
        pass

    def check_drift(self, agent_name: str,
                   k_recent: int = 10) -> DriftReport:
        # Fetch K most recent traces
        # Compute metrics (embeddings, lengths, tokens)
        # Compare to stored baseline
        # Return DriftReport with alert flags
        pass

    def run_continuous(self,
                      check_interval_seconds: int = 300):
        # Background loop: check all agents every interval
        # Sleep between checks to avoid resource waste
```

3.4 Research Angle

This is where AgentTrace becomes a publishable research contribution. The key innovation is a **formal definition of behavioral drift in multi-agent LLM systems** combined with a practical statistical

framework for detecting it.

Research contributions:

- Formal definition of "behavioral drift" in multi-agent LLM systems
- A statistical framework combining embedding similarity + distribution tests
- Empirical evaluation: intentionally drifted agents, measuring precision/recall
- **Target venue:** ACL 2027 Workshop on LLM Evaluation, or MLSys 2027

Deliverable: Drift detector running as a Docker service. Alerts firing correctly in automated tests.

PHASE 4

Phase 4: Dashboard (Weeks 5–7)

Goal: A visual UI that makes traces understandable at a glance. Five pages covering every aspect of agent observability.

4.1 Pages and Views

Page 1 — Traces List

- Table of all recent traces with columns: Trace Name, Agent, Started, Duration, Total Tokens, Total Cost, Status
- Filter by: date range, agent name, model, status (running/completed/error)
- Click any row to navigate to the Trace Detail view

Page 2 — Trace Detail / Timeline View

- Gantt-style timeline showing all spans in a single trace execution
- Color-coded by type: LLM call (blue), tool call (orange), sub-agent (green)
- Hover over any span: shows truncated prompt/response, tokens, cost, latency
- Vertical error markers showing where exceptions occurred
- Expandable JSON viewer for full prompt/response inspection

Page 3 — Cost & Token Analytics

- Line chart: daily/weekly token spend over time
- Bar chart: cost breakdown by agent name
- Bar chart: cost breakdown by model
- "Most expensive traces" table for identifying cost outliers

Page 4 — Drift Monitor

- Line chart: cosine similarity score over time per agent ($y=1.0$ means no drift)
- Alert timeline: when drift was detected, with severity indicators
- Baseline info panel: when baseline was built, on how many traces
- "Rebuild Baseline" button for manual rebaselining

Page 5 — Settings

- Collector URL configuration
- Alert webhook URLs (Slack, Discord, custom)
- SMTP email configuration for email alerts
- Pricing configuration (upload custom pricing.json)

- Data retention period (auto-delete old traces after N days)

4.2 Frontend Technology Stack

Technology	Purpose	Notes
React 18 + TypeScript	UI framework	Type-safe, large ecosystem
Vite	Build tool	Fast HMR, instant dev server
TailwindCSS + shadcn/ui	Styling + components	Clean, accessible components
D3.js	Gantt timeline chart	Custom visualization (not a library chart)
React Query	Data fetching + caching	Auto-refetch, background updates
Zustand	State management	Lightweight, minimal boilerplate

4.3 Dashboard Dockerfile

Multi-stage build: Node.js build stage compiles TypeScript and bundles assets, then an Nginx serve stage serves the static files. Final image size is approximately **25MB**.

Deliverable: Full UI running at `localhost:3000`. All 5 pages functional and connected to the Collector REST API.

PHASE 5

Phase 5: Framework Integrations Expansion (Week 7)

Goal: Support all major agent frameworks out of the box. Every integration should require at most two lines of code to enable.

Framework	Integration Method	Lines to Enable
LangChain	CallbackHandler (Phase 1)	2 lines
LangGraph	StateGraph middleware	2 lines
CrewAI	Agent wrapper patch	2 lines
AutoGen	ConversableAgent patch	2 lines
OpenAI Agents SDK	Tracing hook	2 lines
Ollama (local LLMs)	httpx middleware	2 lines
Raw OpenAI API	httpx middleware	2 lines
Custom agents	Manual span API	~5 lines

Example: CrewAI Integration

CrewAI Integration Example

```
from agenttrace.integrations.crewai import patch_crewai
patch_crewai() # One line patches all CrewAI agents

# Your existing CrewAI code runs unchanged
# All traces are captured automatically
from crewai import Agent, Task, Crew
agent = Agent(role="researcher", goal="...", llm="gpt-4o")
task = Task(description="Research topic X", agent=agent)
crew = Crew(agents=[agent], tasks=[task])
result = crew.kickoff()
```

Deliverable: All framework integrations tested and documented with quickstart guides.

PHASE 6

Phase 6: Documentation & Community Launch (Weeks 8–10)

Goal: Make the project easy to discover, use, and contribute to. A well-documented project attracts contributors; a poorly documented one dies.

6.1 Documentation Site (MkDocs Material)

Documentation structure:

- Getting Started — 5-minute quickstart
- Installation — pip, Docker, from source
- Framework Integrations — one page per framework (LangChain, LangGraph, CrewAI, AutoGen, etc.)
- Configuration Reference — all environment variables and config options
- API Reference — auto-generated from FastAPI OpenAPI schema
- Drift Detection — How It Works — detailed explanation of the algorithm
- Deployment Guide — Docker, bare metal, Render, Railway
- Contributing Guide — development setup, coding standards, PR process
- Changelog — version history

6.2 Demo

- Record a 3-minute screen recording showing AgentTrace tracing a real multi-agent CrewAI pipeline
- Host on YouTube, embed in README and documentation
- Create a live demo instance on Render free tier (optional)

6.3 Community Launch Checklist

- Post on Hacker News: "Show HN: AgentTrace — open-source observability for multi-agent AI"
- Post on Reddit: r/LocalLLaMA, r/aiagents, r/MachineLearning, r/LangChain
- Tweet/post on LinkedIn with dashboard screenshots
- Submit to awesome-langchain, awesome-llm, awesome-ai-agents lists on GitHub
- Open GitHub Discussions for feature requests and community Q&A
- Tag CrewAI, LangGraph, and AutoGen maintainers in GitHub issues requesting official integration
- Post in LangChain Discord, CrewAI Discord, and AutoGen Discord servers

6.4 PyPI Package

- Package name: `agenttrace`
- Install command: `pip install agenttrace`
- Minimal dependencies: `httpx`, `pydantic`
- Optional extras: `pip install agenttrace[drift]` for sentence-transformers

Deliverable: Docs live at GitHub Pages. Package on PyPI. Launch posts published across all channels.

PHASE 7

Phase 7: Research Paper (Months 2.5–3)

Goal: Turn the drift detection method into a publishable research paper. This establishes academic credibility and drives long-term adoption.

7.1 Paper Outline

Title: "AgentTrace: A Framework for Behavioral Drift Detection in Production Multi-Agent LLM Systems"

Abstract (~150 words):

Multi-agent LLM systems are increasingly deployed in production, yet their observability remains fragmented. We present AgentTrace, an open-source framework that provides end-to-end tracing for multi-agent pipelines and introduces a novel behavioral drift detection algorithm. Our method combines embedding cosine similarity with statistical distribution tests to detect when model provider updates silently alter agent behavior. We evaluate our approach on intentionally drifted agents across three drift types (model update, temperature change, prompt mutation) and demonstrate high precision and recall with sub-minute detection latency.

Paper Sections:

- **1. Introduction** — The observability gap in multi-agent systems
- **2. Related Work** — LangSmith, Langfuse, Arize, OpenTelemetry for AI
- **3. System Architecture** — The AgentTrace pipeline
- **4. Behavioral Drift Detection Algorithm** — Formal definition, cosine similarity baseline, KS test, combined score
- **5. Experimental Evaluation** — Dataset, metrics (precision/recall/F1/time-to-detect), baselines
- **6. Case Studies** — Real-world drift scenarios
- **7. Conclusion and Future Work**

7.2 Experiment Setup (RTX 4090 — Zero Cost)

All experiments run locally on your RTX 4090. No API costs required.

- Run Qwen3 14B locally via Ollama as the base model
- Build a test agent with LangGraph (multi-step reasoning + tool use)
- Simulate drift by: changing model temperature, swapping model version, altering system prompt
- Run AgentTrace drift detector and measure detection speed and accuracy
- Write results table with precision, recall, F1, and time-to-detect for each drift type

Drift Type	Simulation Method	Expected Difficulty
Model version swap	Switch from Qwen3-14B to Qwen3-8B mid-run	Medium — subtle semantic changes
Temperature change	Shift temperature from 0.1 to 0.8	Easy — length/variance changes significantly
System prompt mutation	Add/remove instructions	Hard — semantic drift without statistical shift

Target Venue: ACL 2027 Workshop on LLM Evaluation, or arXiv preprint first for community feedback.

5. Full Timeline Summary

Week	Phase	Milestone
Week 1	Phase 0: Setup	Repo structure, CI/CD, README live, first commit
Week 2	Phase 1a: SDK Core	Core tracer class + async event queue working
Week 3	Phase 1b: Integrations	LangChain + LangGraph integration working
Weeks 3–4	Phase 2: Collector	Collector API + SQLite storage, REST API live
Weeks 4–5	Phase 3: Drift Detector	Drift detection service running, alerts working
Weeks 5–6	Phase 4a: Dashboard	Dashboard: Trace list + Timeline view
Weeks 6–7	Phase 4b: Dashboard	Dashboard: Cost analytics + Drift monitor
Week 7	Phase 5: Integrations	All framework integrations (CrewAI, AutoGen, etc.)
Weeks 8–9	Phase 6a: Docs	Documentation site + PyPI publish
Weeks 9–10	Phase 6b: Launch	Community launch across all channels
Months 2.5–3	Phase 7: Research	Research paper experiments + writing + submission

The timeline is designed to be achievable alongside a 6th-semester course load. Each phase has a clear deliverable that can be demonstrated independently. The most important phases (1–3) produce a functional, differentiated tool. Phases 4–6 add polish and community. Phase 7 converts the work into academic output.

6. GitHub Repository Best Practices

6.1 README Structure

The README is the most important page of the project. It must convince a developer to try AgentTrace within 30 seconds of landing on the page.

- **1. Logo + Badges** — PyPI version, license (MIT), tests passing, GitHub stars
- **2. One-Line Description** — "Open-source observability for multi-agent AI systems"
- **3. 30-Second Quickstart** — Code block showing pip install + 2-line usage
- **4. Screenshots** — Dashboard timeline view, cost analytics, drift monitor
- **5. Features List** — Bullet points: tracing, drift detection, cost tracking, self-hosted
- **6. Installation** — pip install, Docker Compose, from source
- **7. Framework Support Table** — Which frameworks are supported and how
- **8. Contributing** — Link to CONTRIBUTING.md
- **9. License** — MIT

6.2 GitHub Labels to Create

Label	Color	Purpose
good first issue	Green	Onboarding tasks for community contributors
bug	Red	Bug reports and fixes
enhancement	Blue	Feature requests and improvements
documentation	Purple	Documentation improvements
integration: langchain	Orange	LangChain-specific issues
integration: crewai	Orange	CrewAI-specific issues
integration: autogen	Orange	AutoGen-specific issues
research	Teal	Paper-related issues and experiments

6.3 First 10 Issues to Open (Before Launch)

Open these as GitHub Issues before the community launch. They signal an active, welcoming project and give new contributors clear entry points.

#	Issue Title	Labels
1	Add AutoGen integration	good first issue, integration
2	Add Mistral API cost table to pricing.json	good first issue
3	Add dark mode to dashboard	enhancement
4	Add export traces to JSON/CSV	enhancement
5	Add PostgreSQL migration guide	documentation
6	Write tutorial: tracing a CrewAI pipeline	documentation
7	Add Prometheus metrics endpoint	enhancement
8	Add trace search/filter by model name	enhancement
9	Add keyboard shortcuts to dashboard	enhancement
10	Add OpenTelemetry export compatibility	enhancement

7. Zero-Cost Infrastructure Setup

7.1 Development Setup

Get the entire stack running in under 2 minutes:

```
Development Quick Start

# 1. Clone the repository
git clone https://github.com/umairinayat/agenttrace
cd agenttrace

# 2. Start everything (collector + dashboard + drift detector)
docker compose up

# 3. Install the SDK in development mode
pip install -e ./sdk

# 4. Test with a sample agent
python examples/langchain_example.py

# 5. Open the dashboard
```

7.2 Free Cloud Services

Service	Purpose	Free Tier
GitHub	Code hosting, CI/CD, Issues, Discussions	Free for public repos (unlimited)
PyPI	Python package hosting	Free (unlimited packages)
GitHub Pages	Documentation hosting (MkDocs)	Free for public repos
Render.com	Live demo hosting	750 hours/month free
DockerHub	Docker image registry	Free for public images
GitHub Releases	Binary/artifact distribution	Free (unlimited)

7.3 Local LLMs for Testing (RTX 4090)

Your RTX 4090 with 24GB VRAM can run these models locally via Ollama — zero API cost:

Model	VRAM Required	Ideal For
Qwen3 14B	~10GB	Primary testing model, excellent reasoning
Llama 4 Scout	~16GB	Multi-modal agent testing
Mistral 7B	~5GB	Fast iteration, lightweight testing
Phi-3 Medium	~8GB	Microsoft ecosystem testing

Ollama Setup

```
# Install Ollama (free, open source)
curl -fsSL https://ollama.ai/install.sh | sh

# Pull a model (one-time download)
ollama pull qwen3:14b

# AgentTrace works with Ollama out of the box
```

8. Success Metrics

8.1 Three-Month Goals

Metric	Target	Why It Matters
GitHub Stars	500+	Social proof, discoverability in trending
PyPI Downloads	1,000+/month	Adoption indicator, dependency potential
Framework Integrations	6	LangChain, LangGraph, CrewAI, AutoGen, OpenAI SDK, Ollama
Documentation Pages	20+	Completeness signals professionalism
External Contributors	5+	Community-driven development momentum
arXiv Preprint	Submitted	Academic credibility, citation potential

8.2 What Success Looks Like

- Framework maintainers (LangGraph, CrewAI) mention AgentTrace in their official documentation or integration guides
- Developers posting screenshots of their traces on Reddit and Twitter/X, organically
- Issues being opened by users you do not know — the strongest signal of real adoption
- Citations in other academic papers studying LLM agent observability or behavioral drift
- Companies evaluating AgentTrace as an alternative to commercial solutions (Arize AI, LangSmith)
- Conference or workshop talk invitations based on the research paper

AgentTrace

Document Version: 1.0 — March 2026

Repository: github.com/umairinayat/agenttrace (to be created)

License: MIT — Free to use, share, and build on.

"Build it once. Let the community make it great."